

LogPulse AI - Incident Report

Generated : 2026-05-30 20:15:51 UTC

Trace ID : 9fa1ccbc-46fc-4fd1-9a9d-3a6725f97934

1 - Incident Summary

Incident ID : 9fa1ccbc
Category : Memory
Severity : SeverityLevel.CRITICAL
Source : Kubernetes
Cache : MISS

2 - Raw Log

OOMKilled: pod/worker-api exceeded memory limit 256Mi

3 - Classification

Category : Memory
Severity : SeverityLevel.CRITICAL
Summary : Kubernetes pod killed for exceeding its memory limit.
Source : Kubernetes

4 - Diagnosis

Root Cause : The worker-api pod was configured with an insufficient memory limit (256Mi) for its operational workload, causing it to be OOMKilled by Kubernetes.

Confidence : 95%

Reasoning : The raw log explicitly states 'OOMKilled: pod/worker-api exceeded memory limit 256Mi', which is directly supported by the classification and the highly confident playbook titled 'Pod OOMKilled: Insufficient Memory Limit'.

Source Tier : llm
Used History : No
Used Playbook : Yes

5 - Proposed Solution

Explanation:

The worker-api pod in the production namespace was terminated because its memory usage exceeded the configured limit of 256Mi.

Steps:

- 1. Retrieve the detailed status of the worker-api pod to confirm the OOMKilled termination reason.
- 2. Inspect the logs of the terminated worker-api container to identify memory-intensive operations prior to the crash.
- 3. Check the current CPU and memory utilization of the worker-api pod.
- 4. Increase the deployment's memory limit to 512Mi and set memory request to 256Mi to prevent future OOM kills.
- 5. Monitor the deployment rollout to ensure the new pods with updated memory limits start successfully.

Commands:

```
$ kubectl get pod worker-api -n production -o yaml
$ kubectl logs worker-api -n production --previous --tail=100
$ kubectl top pod worker-api -n production
$ kubectl patch deployment worker-api -n production -p
'{"spec":{"template":{"spec":{"containers":[{"name":"worker-api","resources":{"requests":{"memory":"256Mi"},"limits":{"memory":"512Mi"}}}]}}}}'
$ kubectl rollout status deployment/worker-api -n production
```

6 - Risk Assessment

Risk Level : RISKLEVEL.SAFE

Safe Commands (5):

```
[OK] kubectl get pod worker-api -n production -o yaml
[OK] kubectl logs worker-api -n production --previous --tail=100
[OK] kubectl top pod worker-api -n production
[OK] kubectl patch deployment worker-api -n production -p
'{"spec":{"template":{"spec":{"containers":[{"name":"worker-api","resources":{"requests":{"memory":"256Mi"},"limits":{"memory":"512Mi"}}}]}}}}'
[OK] kubectl rollout status deployment/worker-api -n production
```

Flagged Commands (0):

(none)

7 - Operator Decision

Decision : OPERATORDECISION.APPROVED

Executed At : 2026-05-30T17:15:51.186610+00:00

Sandbox Results:

Command : kubectl get pod worker-api -n production -o yaml

Exit : -1

Time : 0ms

Output:

[sandbox unavailable -- docker not found]

Command : kubectl logs worker-api -n production --previous --tail=100

Exit : -1

Time : 0ms

Output:

[sandbox unavailable -- docker not found]

Command : kubectl top pod worker-api -n production

Exit : -1

Time : 0ms

Output:

[sandbox unavailable -- docker not found]

Command : kubectl patch deployment worker-api -n production -p
'{"spec":{"template":{"spec":{"containers":[{"name":"worker-api","resources":{"requests":{"memory":"256Mi"},"limits":{"memory":"512Mi"}}}]}}}}'

Exit : -1

Time : 0ms

Output:

[sandbox unavailable -- docker not found]

Command : kubectl rollout status deployment/worker-api -n production

Exit : -1

Time : 0ms

Output:

[sandbox unavailable -- docker not found]

8 - Historical Context

Times Seen : 1

First occurrence -- no prior incidents.

9 - Incident Timeline

[17:15:51] WORKFLOW_COMPLETED -> Completed in 78202ms